Week 4, Notebook 2: Graph Neural Networks with PyTorch

From Scratch to Framework — GCN, GraphSAGE, and Real Datasets

What you'll build: GNNs on real graph datasets using PyTorch, exploring different aggregation strategies.

New concepts:

- GCN vs GraphSAGE vs GAT (different message-passing flavors)
- Node classification on real citation networks
- Graph-level pooling for graph classification
- Over-smoothing: why very deep GNNs fail

Time estimate: 60 minutes

In [1]

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(42)
np.random.seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")
```

Part 1: GCN in Pure PyTorch (No Geometric Needed)

In [2]

```python
# ============================================================
# GCN layer in PyTorch — same math as Week 4 Notebook 1
# ============================================================

class GCNConv(nn.Module):
    """Graph Convolutional layer (Kipf & Welling, 2017)."""

    def __init__(self, in_features, out_features):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features, bias=True)
        nn.init.kaiming_normal_(self.linear.weight, nonlinearity='relu')

    def forward(self, X, A_norm):
        """
        X: node features (N x F_in)
        A_norm: normalized adjacency (N x N)
        """
        # Message passing + transform
        support = self.linear(X)     # Transform: X @ W + b
        out = A_norm @ support       # Aggregate: multiply by normalized adjacency
        return out


class GCNNet(nn.Module):
    """2-layer GCN for node classification."""

    def __init__(self, in_dim, hidden_dim, out_dim, dropout=0.5):
        super().__init__()
        self.conv1 = GCNConv(in_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, out_dim)
        self.dropout = dropout

    def forward(self, X, A_norm):
        h = self.conv1(X, A_norm)
        h = F.relu(h)
        h = F.dropout(h, p=self.dropout, training=self.training)
        h = self.conv2(h, A_norm)
        return h  # raw logits; use F.log_softmax for loss


def normalize_adj_torch(A):
    """Symmetric normalization: D^{-1/2} A_hat D^{-1/2}."""
    A_hat = A + torch.eye(A.size(0), device=A.device)
    D = A_hat.sum(dim=1)
    D_inv_sqrt = torch.diag(1.0 / torch.sqrt(D))
    return D_inv_sqrt @ A_hat @ D_inv_sqrt

print("GCN classes defined. Ready for training!")
```

Part 2: Generate a Larger Graph Dataset

In [3]

```python
# ============================================================
# Generate a Stochastic Block Model graph (2 communities)
# ============================================================

def generate_sbm(n_nodes=200, n_classes=3, p_intra=0.15, p_inter=0.01, feat_dim=16):
    """Generate a graph with community structure."""
    labels = np.repeat(np.arange(n_classes), n_nodes // n_classes)
    n = len(labels)

    # Build adjacency: high prob within communities, low between
    A = np.zeros((n, n))
    for i in range(n):
        for j in range(i+1, n):
            p = p_intra if labels[i] == labels[j] else p_inter
            if np.random.rand() < p:
                A[i, j] = A[j, i] = 1

    # Node features: noisy cluster centers
    centers = np.random.randn(n_classes, feat_dim) * 2
    X = np.array([centers[labels[i]] + np.random.randn(feat_dim) * 0.8 for i in range(n)])

    return torch.FloatTensor(A), torch.FloatTensor(X), torch.LongTensor(labels)

A, X, y = generate_sbm(300, n_classes=3, feat_dim=16)
A_norm = normalize_adj_torch(A)

# Train/val/test split (random)
n = len(y)
perm = torch.randperm(n)
train_mask = torch.zeros(n, dtype=torch.bool)
val_mask = torch.zeros(n, dtype=torch.bool)
test_mask = torch.zeros(n, dtype=torch.bool)
train_mask[perm[:int(0.6*n)]] = True
val_mask[perm[int(0.6*n):int(0.8*n)]] = True
test_mask[perm[int(0.8*n):]] = True

print(f"Graph: {n} nodes, {int(A.sum()/2)} edges, {len(torch.unique(y))} classes")
print(f"Features per node: {X.shape[1]}")
print(f"Split: {train_mask.sum()} train, {val_mask.sum()} val, {test_mask.sum()} test")
```

Part 3: Train and Evaluate

In [4]

```
# ============================================================
# Training loop for node classification
# ============================================================

model = GCNNet(in_dim=16, hidden_dim=32, out_dim=3, dropout=0.5)
optimizer = optim.Adam(model.parameters(), lr=0.01, weight_decay=5e-4)

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(300):
    # Train
    model.train()
    logits = model(X, A_norm)
    loss = F.cross_entropy(logits[train_mask], y[train_mask])

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Eval
    model.eval()
    with torch.no_grad():
        logits = model(X, A_norm)
        train_loss = F.cross_entropy(logits[train_mask], y[train_mask]).item()
        val_loss = F.cross_entropy(logits[val_mask], y[val_mask]).item()
        train_acc = (logits[train_mask].argmax(1) == y[train_mask]).float().mean().item()
        val_acc = (logits[val_mask].argmax(1) == y[val_mask]).float().mean().item()

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    if epoch % 50 == 0:
        print(f"  Epoch {epoch:3d} | Train Loss: {train_loss:.4f} | "
              f"Val Acc: {val_acc:.3f}")

# Test
model.eval()
with torch.no_grad():
    test_logits = model(X, A_norm)
    test_acc = (test_logits[test_mask].argmax(1) == y[test_mask]).float().mean().item()
print(f"\nTest Accuracy: {test_acc:.1%}")
```

In [5]

```
# Visualize training and learned embeddings
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].plot(history['train_loss'], label='Train', alpha=0.8)
axes[0].plot(history['val_loss'], label='Val', alpha=0.8)
axes[0].set_title('Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.2)

axes[1].plot(history['train_acc'], label='Train', alpha=0.8)
axes[1].plot(history['val_acc'], label='Val', alpha=0.8)
axes[1].set_title('Accuracy')
axes[1].set_ylim(0, 1.05)
axes[1].legend()
axes[1].grid(True, alpha=0.2)

# Node embeddings from hidden layer
model.eval()
with torch.no_grad():
    h1 = model.conv1(X, A_norm)
    h1 = F.relu(h1)
    h_np = h1.numpy()
    y_np = y.numpy()

# Simple 2D projection (first 2 dims of hidden)
colors = ['steelblue', 'coral', 'forestgreen']
for cls in range(3):
    mask = y_np == cls
    axes[2].scatter(h_np[mask, 0], h_np[mask, 1], c=colors[cls],
                    s=20, alpha=0.6, label=f'Class {cls}')
axes[2].set_title('Learned Node Embeddings')
axes[2].legend()
axes[2].grid(True, alpha=0.2)

plt.suptitle('GCN ON COMMUNITY GRAPH', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('w4_02_gcn_pytorch.png', dpi=100, bbox_inches='tight')
plt.show()
```

Part 4: GraphSAGE — A Different Aggregation Strategy

GCN uses **spectral** convolution (fixed normalized adjacency). GraphSAGE uses **sampling + aggregation** — more scalable.

Key difference: instead of multiplying by the full normalized adjacency, GraphSAGE **samples** a fixed number of neighbors and aggregates with mean/max/LSTM.

In [6]

```python
# ============================================================
# GraphSAGE-style layer (mean aggregation)
# ============================================================

class SAGEConv(nn.Module):
    """Simplified GraphSAGE layer (mean aggregation)."""

    def __init__(self, in_features, out_features):
        super().__init__()
        # Separate transforms for self and neighbors
        self.W_self = nn.Linear(in_features, out_features, bias=False)
        self.W_neigh = nn.Linear(in_features, out_features, bias=True)
        nn.init.kaiming_normal_(self.W_self.weight)
        nn.init.kaiming_normal_(self.W_neigh.weight)

    def forward(self, X, A):
        # Neighbor aggregation (mean)
        A_hat = A + torch.eye(A.size(0), device=A.device)
        D = A_hat.sum(dim=1, keepdim=True)
        neigh_mean = (A_hat @ X) / D  # Mean of neighbors (+ self)

        # Combine self + neighbor representations
        h_self = self.W_self(X)
        h_neigh = self.W_neigh(neigh_mean)
        return h_self + h_neigh


class SAGENet(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim, dropout=0.5):
        super().__init__()
        self.sage1 = SAGEConv(in_dim, hidden_dim)
        self.sage2 = SAGEConv(hidden_dim, out_dim)
        self.dropout = dropout

    def forward(self, X, A):
        h = F.relu(self.sage1(X, A))
        h = F.dropout(h, p=self.dropout, training=self.training)
        h = self.sage2(h, A)
        return h


# Train GraphSAGE
sage = SAGENet(16, 32, 3)
optimizer = optim.Adam(sage.parameters(), lr=0.01, weight_decay=5e-4)

sage_accs = []
for epoch in range(300):
    sage.train()
    logits = sage(X, A)  # Note: uses raw A, not A_norm
    loss = F.cross_entropy(logits[train_mask], y[train_mask])
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    sage.eval()
    with torch.no_grad():
        val_acc = (sage(X, A)[val_mask].argmax(1) == y[val_mask]).float().mean().item()
        sage_accs.append(val_acc)
```

```
59
60 sage.eval()
61 with torch.no_grad():
62     test_acc_sage = (sage(X, A)[test_mask].argmax(1) == y[test_mask]).float().mean().item()
63
64 print(f"GraphSAGE Test Accuracy: {test_acc_sage:.1%}")
65 print(f"GCN Test Accuracy:       {test_acc:.1%}")
```

Part 5: Over-Smoothing — Why Depth Hurts in GNNs

In standard deep learning, more layers = more power. In GNNs, too many layers causes **over-smoothing**: all node representations converge to the same value.

Each layer mixes neighbor features → after K layers, every node has seen the entire graph → all embeddings become identical.

In [7]

```python
# ============================================================
# EXPERIMENT: Over-smoothing with increasing depth
# ============================================================

def train_gcn_depth(depth, X, A_norm, y, train_mask, test_mask, epochs=300):
    layers = []
    dims = [16] + [32] * (depth - 1) + [3]
    for i in range(len(dims) - 1):
        layers.append(GCNConv(dims[i], dims[i+1]))

    params = []
    for l in layers:
        params.extend(l.parameters())
    optimizer = optim.Adam(params, lr=0.01, weight_decay=5e-4)

    for epoch in range(epochs):
        for l in layers:
            l.train()
        h = X
        for i, l in enumerate(layers[:-1]):
            h = F.relu(l(h, A_norm))
            h = F.dropout(h, p=0.5, training=True)
        h = layers[-1](h, A_norm)
        loss = F.cross_entropy(h[train_mask], y[train_mask])
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    for l in layers:
        l.eval()
    with torch.no_grad():
        h = X
        for i, l in enumerate(layers[:-1]):
            h = F.relu(l(h, A_norm))
        h = layers[-1](h, A_norm)
        acc = (h[test_mask].argmax(1) == y[test_mask]).float().mean().item()
    return acc

depths = [1, 2, 3, 4, 6, 8, 12]
depth_accs = []
print("Depth experiment:")
for d in depths:
    torch.manual_seed(42)
    acc = train_gcn_depth(d, X, A_norm, y, train_mask, test_mask)
    depth_accs.append(acc)
    print(f"  Depth {d:2d}: Test Acc = {acc:.3f}")

plt.figure(figsize=(8, 4))
plt.plot(depths, depth_accs, 'o-', color='steelblue', markersize=8, linewidth=2)
plt.xlabel('Number of GCN Layers')
plt.ylabel('Test Accuracy')
plt.title('OVER-SMOOTHING: Deeper GNNs Perform Worse')
plt.grid(True, alpha=0.2)
plt.ylim(0, 1.05)
plt.savefig('w4_02_oversmoothing.png', dpi=100, bbox_inches='tight')
plt.show()

print("\nKEY INSIGHT: 2-3 layers is the sweet spot for most GNNs.")
```

```python
print("Unlike CNNs/Transformers, deeper is NOT better for graphs.")
```

■ Week 4 Complete — Self-Assessment

You should now be able to:

- [ ] Build a GCN from scratch using adjacency matrix multiplication
- [ ] Explain message passing: "aggregate neighbors, transform, activate"
- [ ] Compare GCN vs GraphSAGE (spectral vs sampling-based)
- [ ] Explain over-smoothing and why 2-3 layers is typical for GNNs
- [ ] Train a GNN for node classification in PyTorch

Key Takeaway:

GNNs generalize neural networks to **irregular structures**. The core idea — message passing — is the same as convolution, just on graphs instead of grids.

➡■ Week 5: Transformers!

Open `W5_01_Pure_Attention_Transformer.ipynb`